

School of Engineering and Technology, MGM University
Chh. Sambhajinagar

Academic Year 2026 - 2027

Mini Project Report
On

"Contact Management System"

Submitted in partial fulfillment for Fifth semester of B.Tech Regular in

COMPUTER SCIENCE AND ENGINEERING
(IoT, Cyber Security Including Block Chain Technology)

By

Nikhil Kadale **202456108054**

Abhishek Yadav **202456108055**

Under the guidance of

Ms. Priyanka Mam
(Asst. Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that **Nikhil Kadale, Abhishek Yadav** with PRN No. **202456108054, 202456108055** has successfully completed their Mini-Project entitled "**Contact Management System**" in the Course of **"Database Management System"** in the Sixth Semester during tenure of completing the B. Tech Regular program in **Computer Science and Engineering (IoT, Cyber Security Including Blockchain Technology)** from **School of Engineering and Technology, MGM University** institute.

Ms. Priyanka Mam
Guide
CSE Department

Ms. Madhuri Kawarkhe
HOD
CSE Department

CONTENT

Sr. No	Title	Page No.
1	Abstract	01
2	Introduction	01
3	Course Outcomes Integrated	01
4	Action Plan	02
5	Literature Review	02
6	Proposed Methodology	02
7	Implementation	03
8	Outputs of the Mini-Project	05
9	Skill Developed / Learning out of this Mini-Project	06
10	Applications of this Mini-Project	07
11	Area of Future Improvement	07
12	Conclusion	07

1. Abstract

Peoplebook is a web-based Contact Management System that allows a user to save, search, edit and delete personal contacts in a secure online address book. Each contact stores a first name, last name, phone number, email address and city. The user signs in with an account, and every contact is stored in a cloud PostgreSQL database provided through Supabase, so the address book remains available from any device with a browser.

The application provides a simple contact list with a live search box, an Add Contact form, and edit and delete options on every contact card. Row Level Security (RLS) policies in the database make sure that each user can see and change only their own contacts. The mini-project demonstrates core relational database concepts such as table design, primary and unique keys, foreign keys, INSERT, SELECT, UPDATE and DELETE statements, pattern searching with LIKE, sorting, aggregate functions and access control. A companion SQL script (contact_management.sql) contains the same add, search and delete operations for practice in a local SQL environment.

2. Introduction

A contact manager stores the details of the people a user needs to reach and lets the user find them quickly. Paper diaries and scattered phone entries are easy to lose and slow to search. Peoplebook replaces them with a structured database so that every contact is saved once, in a fixed format, and can be retrieved by name, phone number or email in a moment.

The project shows how a Database Management System supports a real-world application. Data is organised into related tables: the contacts table holds the address-book entries, and the profiles table holds the display name of each registered user. Each contact record carries a user identifier that links it to the account that owns it, which is the relationship used to keep one user's data separate from another's.

The main objective is to design and demonstrate a structured database for a contact management system and to connect it to a working website. The system uses SQL operations to add a new contact, search contacts by any field, update an existing contact, delete a contact and count the saved records.

Project title: Peoplebook – Contact Management System. **Team members:** Nikhil Kadale (PRN 202456108054) and Abhishek Yadav (PRN 202456108055).

3. Course Outcomes Integrated

The following course outcomes are integrated with this mini-project:

- Understand the role and architecture of a DBMS in a real-world application.
- Design relational tables and identify primary-key and foreign-key relationships.
- Apply SQL commands such as CREATE, INSERT, SELECT, UPDATE, DELETE, WHERE, ORDER BY and LIKE.

- Use aggregate functions, constraints, unique keys and views to keep data correct and easy to report.
- Understand data integrity, user-level access control and secure storage through a database-backed application.

4. Action Plan

S. No.	Details of activity	Planned Start date	Planned Finish date	Name of Responsible Team Members
1	Requirement analysis and feature identification	20/09/2026	22/09/2026	Nikhil Kadale, Abhishek Yadav
2	Database entities and relational schema planning	24/09/2026	27/09/2026	Nikhil Kadale, Abhishek Yadav
3	Website interface and navigation development	27/09/2026	28/09/2026	Nikhil Kadale
4	Database setup and SQL query preparation	29/09/2026	30/09/2026	Abhishek Yadav
5	Add, search, edit, delete and login integration	01/10/2026	03/10/2026	Nikhil Kadale, Abhishek Yadav
6	Testing, screenshots and report preparation	04/10/2026	07/10/2026	Nikhil Kadale, Abhishek Yadav

5. Literature Review

Peoplebook follows a database-backed web application approach. The user interface lets a signed-in user view all saved contacts as cards, type in a search box to filter them, open a form to add a new contact, and use the pencil and bin icons on each card to edit or delete it. A total count of the contacts is shown at the top of the page. The deployed application stores its data in Supabase, which provides a PostgreSQL database, user authentication and access policies as a cloud service.

The database is organised around two related entities. A user (profile) can own many contacts, while each contact belongs to exactly one user. The user identifier stored in every contact row connects it to its owner and avoids duplicating account details. Constraints such as NOT NULL on the first name and phone number keep incomplete records out of the table. These concepts, along with Row Level Security, help maintain data integrity and keep each person’s address book private.

6. Proposed Methodology

The methodology starts by identifying the main user and the workflows: a user registers or logs in, adds contacts, searches for a contact, edits details and deletes contacts that are no longer

needed. The entities and their relationships are then represented as relational tables, and each SQL statement is mapped to one function of the website.

The database contains a contacts table (id, user_id, first_name, last_name, phone, email and related fields) and a profiles table (id, display_name, created_at, updated_at). The id of a profile is the same as the identifier of the authenticated user, and the user_id column of a contact refers to that user.

The application adds records with INSERT, retrieves them with SELECT, and narrows results with WHERE, LIKE and ORDER BY. A search keyword is matched against the name, phone number and email of every contact, so one query serves the whole search box. Contacts are changed with UPDATE and removed with DELETE, and the COUNT aggregate function supplies the number of saved contacts shown on the page.

For security, Row Level Security is enabled on the table. A policy compares the logged-in user's identifier with the user_id of each row, so every SELECT, INSERT, UPDATE and DELETE is automatically limited to the user's own contacts.

7. Implementation

7.1 Frontend

The web interface, named Peoplebook, has a header with the application name, a light/dark theme toggle and a Log out button. The main page greets the user, shows the heading "Your contacts" with the number of people in the address book, a search box ("Search name, number, email…") and an Add contact button. Each contact is shown as a card with the initials of the person, the full name, city, phone number and email, together with edit and delete icons. The interface is built with React, TypeScript and Tailwind CSS and was generated and refined using the Lovable app builder.

7.2 Database and backend services

The website uses a Supabase PostgreSQL database. Supabase provides the cloud database, the login system and the Row Level Security policies, while SQL defines how the contact data is stored and queried. The table structure used by the application is summarised below.

Column	Data type	Description
id	uuid (primary key)	Unique identifier of each contact, generated automatically
user_id	uuid (foreign key)	Owner of the contact; refers to the logged-in user
first_name	text, NOT NULL	First name of the contact
last_name	text	Last name of the contact
phone	text, NOT NULL	Phone number with country code
email	text	Email address (optional)
city	text	City shown on the contact card

The profiles table stores id (uuid), display_name (text), created_at and updated_at (timestamps) for every registered user.

7.3 SQL implementation

The statements below are the PostgreSQL form of the operations used by the application. The complete script, written for SQLite so it can be run directly, is supplied as contact_management.sql.

```
-- Create the table
CREATE TABLE contacts (
  id         uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    uuid NOT NULL DEFAULT auth.uid()
             REFERENCES auth.users(id) ON DELETE CASCADE,
  first_name text NOT NULL,
  last_name  text,
  phone      text NOT NULL,
  email      text,
  city       text,
  created_at timestamptz DEFAULT now(),
  UNIQUE (user_id, phone)
);
```

Add a contact: a new row is inserted when the Add contact form is submitted.

```
INSERT INTO contacts (first_name, last_name, phone, email, city)
VALUES ('Alex', 'Morgan', '+1-202-555-0101',
        'alex.morgan@example.com', 'Seattle');
```

Search contacts: one keyword is matched against name, phone, email and city, and results are sorted by first name.

```
SELECT * FROM contacts
WHERE first_name ILIKE '%sam%' OR last_name ILIKE '%sam%'
   OR phone ILIKE '%sam%'      OR email ILIKE '%sam%'
   OR city ILIKE '%sam%'
ORDER BY first_name;
```

Edit, delete and count: the pencil icon runs UPDATE, the bin icon runs DELETE, and COUNT gives the total shown on the page.

```
UPDATE contacts SET phone = '+1-202-555-0199' WHERE id = '<contact-id>';
DELETE FROM contacts WHERE id = '<contact-id>';
SELECT COUNT(*) AS total_contacts FROM contacts;
```

Access control: Row Level Security limits every operation to the owner of the row.

```
ALTER TABLE contacts ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Users manage their own contacts" ON contacts
  FOR ALL USING (auth.uid() = user_id)
  WITH CHECK (auth.uid() = user_id);
```

7.4 Testing

The main workflows were checked by signing in, adding a new contact, typing part of a name in the search box, editing a saved contact, deleting a contact and logging out. The Supabase table editor was used to confirm that each action changed the stored rows as expected, and the SQL statements were run against the sample records to verify their results.

8. Outputs of the Mini-Project

The project output is a working Contact Management System named Peoplebook. A user can log in, view the address book, search contacts instantly, add new contacts, edit existing details and delete contacts. The footer of the page states that the address book is private to the user's account, which is enforced by Row Level Security in the database.

The database component consists of the contacts table and the profiles table. The contacts table holds seven records, including the five sample contacts (Alex Morgan, Sam Rivera, Jordan Lee, Taylor Brooks and Casey Patel) and contacts added by the team during testing. The screenshots below document the application and the database.

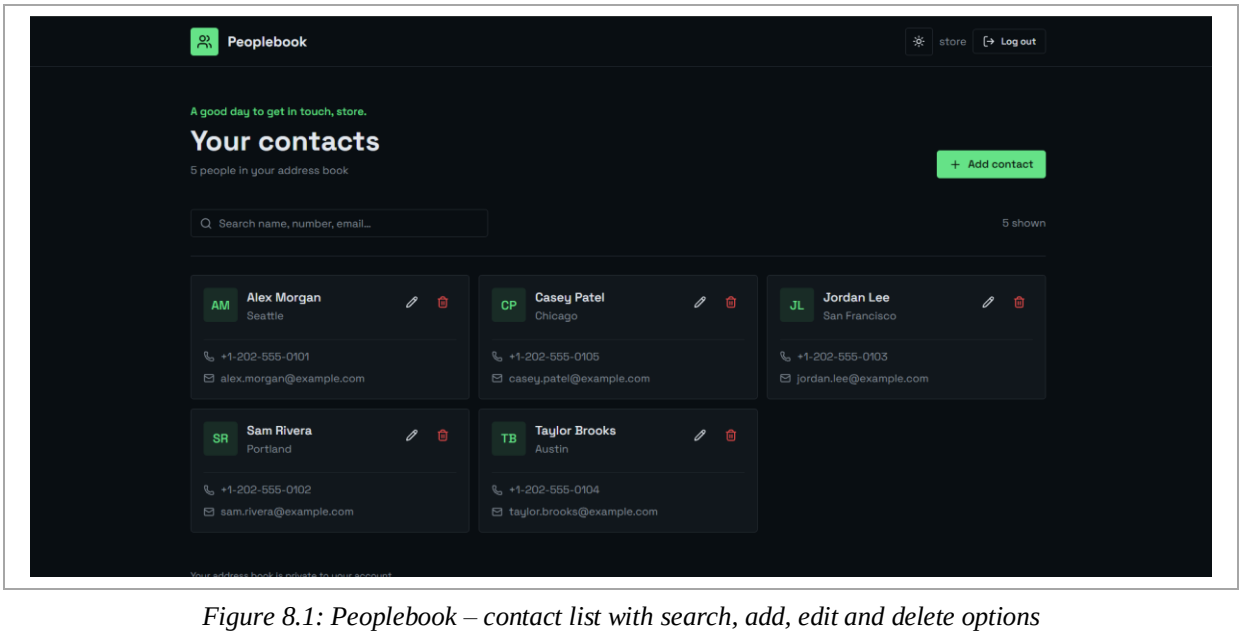


Figure 8.1: Peoplebook – contact list with search, add, edit and delete options

Figure 8.1 shows the main page in dark mode. The header contains the theme toggle and the Log out button. Below the greeting are the Add contact button, the search box and a count of the contacts shown. Each card displays the initials, name, city, phone number and email, with edit and delete icons.

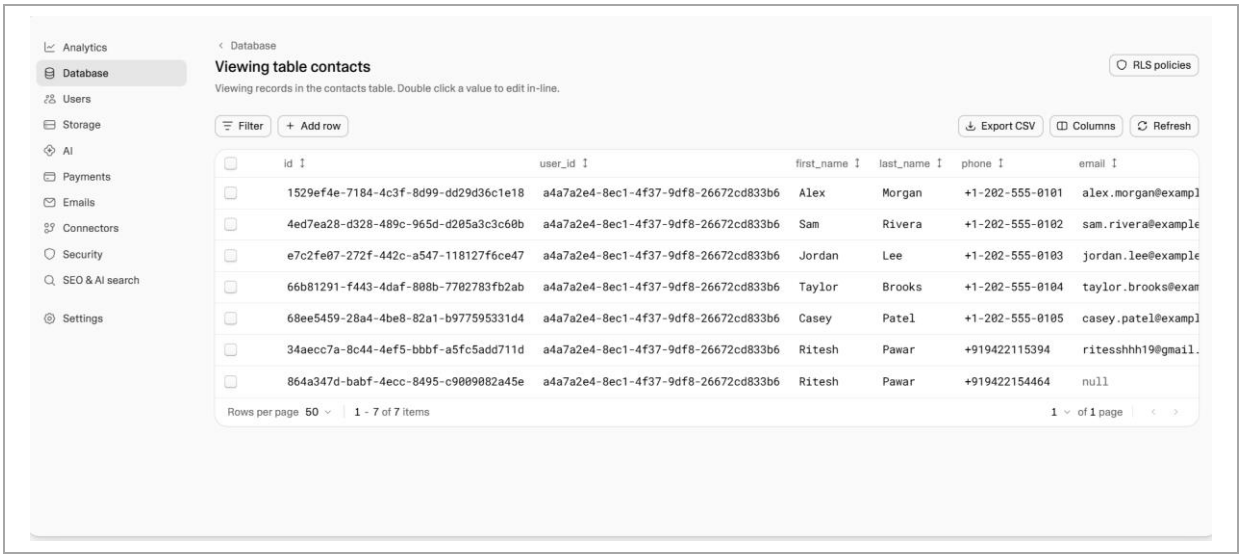


Figure 8.2: Supabase database – contacts table

Figure 8.2 shows the contacts table in the Supabase database viewer with the columns id, user_id, first_name, last_name, phone and email. All rows carry the same user_id, which links them to the account that owns the address book. A contact with no email is stored as null, showing that the email field is optional.



Figure 8.3: Supabase database – profiles table

Figure 8.3 shows the profiles table, which stores the id of the registered user, the display name and the created_at and updated_at timestamps. The id in this table matches the user_id used in the contacts table.

9. Skill Developed / Learning out of this Mini-Project

Through this mini-project, we learned how a DBMS supports a real-world contact management workflow. We practised relational database design, primary, unique and foreign keys, NOT NULL constraints, and the SQL commands INSERT, SELECT, UPDATE and DELETE. We also learned to search with LIKE and ILIKE, to sort results, to count records with aggregate

functions and to protect data with Row Level Security. In addition, we learned how a web application communicates with a cloud database and how user login is connected to the data a person is allowed to see.

10. Applications of this Mini-Project

Peoplebook can be used as a personal address book, and the same design applies to many other settings: customer and client lists for small businesses, student or alumni directories, a staff phone directory for an office, and the contact section of a CRM. The database concepts used here, namely structured storage, quick searching and per-user access control, are relevant to any application that stores personal records.

11. Area of Future Improvement

Future improvements could include contact groups and labels such as family, friends and work, a favourites option, profile photos, additional phone numbers and addresses, and birthday reminders. Contacts could also be imported from and exported to CSV or vCard files, and duplicate contacts could be detected and merged.

On the database side, indexes can be added to the name, phone and email columns to keep searches fast as the data grows, and fuller validation rules can be added for phone and email formats. Automated tests can be written for the add, search, edit and delete workflows. Any production deployment should review authentication, permissions, backups and the protection of personal contact information.

12. Conclusion

Peoplebook demonstrates how a relational DBMS can support a contact management system from adding a contact to searching, editing and deleting it. The project connects each feature of the website with a database operation and illustrates SQL concepts such as constraints, filtering with LIKE, sorting, aggregate functions and Row Level Security. Supabase and PostgreSQL are used by the deployed application, while the accompanying SQL script makes the same operations easy to practise in a local SQL environment. The project helped us understand the practical role of database design, data integrity and queries in web application development.